

Django 教學 07

資料列表搜尋與分頁

第 12～13 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [06_ORM與Admin](#)。

- **第 12 章：資料驅動頁面**
- **第 13 章：CBV 與列表整合**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 12 章

資料驅動頁面

用 Function View 追蹤列表與詳情，完成搜尋、分類及空結果檢核。

本章的操作環境與成果

前台實作選 LearnBoard 或 LearnMart；圖解與最小範例先說明概念，再對照現有程式。

完成成果： 用 Function View 追蹤列表與詳情，完成搜尋、分類及空結果檢核。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

12-1 前台整合前的資料層檢核

檢核問題	回查位置
pk 與外鍵如何定位、連結資料？	第 6、8 章
刪除分類、商品與作者會影響誰？	第 8 章
修改 Model 後何時需要 migration？	第 9 章
查詢回傳 instance 還是 QuerySet？	第 10 章
如何在 Admin 建立並搜尋資料？	第 11 章

拿出自己的 shell、migration 與 Admin 操作結果說明，不再建立另一套專案。

12-2 整合準備：後台資料與前台頁面

切回 LearnMart 或 LearnBoard，確認使用的是同一個專案環境與資料庫。

1. 依第 2 章啟動流程執行既有 migration，啟動伺服器。
2. 用已建立的管理帳號登入 `/admin/`。
3. 在既有商品或留言後台，新增或確認一筆可辨識的練習資料。
4. 開啟前台列表，確認能找到這筆資料。

商品請確認上架與篩選條件；查不到時，先比對 View 的查詢條件。
後台尚未設定完成時，回第 11 章完成註冊與管理帳號設定。

12-3 從一筆後台資料追蹤前台

選一個現有專案實作，另一個用來比較。

專案	追蹤路徑	驗收
LearnBoard	Message、列表 View、留言模板	找到留言並搜尋
LearnMart	Product、列表 View、商品卡	找到商品並開啟詳情

本章先以 Function View 看清查詢與 render，第 13 章再讀現有 CBV。
catalog 練習模型沒有 `is_active`、`seller`；現有專案片段不直接貼進 catalog。

12-4 專案對照實作：先找檔案，再說明理由

1. 分別啟動兩個專案，記錄環境同步、migration、seed、預覽結果。
2. 找出 `config/urls.py`、app 的 `urls.py` 與首頁 View。
3. 在兩個專案各用 shell 執行首頁的 `reverse()`，確認皆為 `/`。
4. 說明相同 `/` 為何顯示不同頁面，以及各自用了哪份 settings。
5. 比較 TOML、app 註冊、User、media、context processor。
6. 在新資料庫比對示範資料筆數，重跑 seed 後說明哪些值不會重設。

依序測試時先用 `Ctrl+C` 停止前一個伺服器。

若要同時看兩個網站，可讓第二個使用 8001，注意 cookie 仍可能互相影響。

12-5 先用 Function View 看清全部責任

教學用最小範例

```
from django.shortcuts import render
from .models import Product

def product_list(request):
    products = Product.objects.filter(is_active=True)
    return render(request, "marketplace/home.html", {
        "products": products,
    })
```

這不是目前 LearnMart 最終 class-based view；它先讓 beginner 看見 query、context 與 render 在同一函式中的資料流。

12-6 List 與 Detail 是兩種不同查詢形狀

List

```
Product.objects.filter(is_active=True)
```

回 QuerySet，可能 0 到多筆。

Detail

```
get_object_or_404(Product, pk=pk, is_active=True)
```

回一個 instance；找不到時回 404。

URL 也對應：

/	商品列表
/products/<int:pk>/	單一商品

12-7 `get_object_or_404` 做了什麼？

```
product = get_object_or_404(  
    Product,  
    pk=pk,  
    is_active=True,  
)
```

可理解為：

1. 用條件查詢一筆物件
2. 找到：回傳 Product instance
3. 找不到：拋出 `Http404`，由 Django 形成 404 response

這比讓 `DoesNotExist` 變成 500 更符合「網址或可見物件不存在」的語意。

權限章還會使用 `scoped queryset` 讓無權物件同樣回 404。

12-8 GET 搜尋表單先建立 URL 狀態

```
<form action="{% url 'marketplace:home' %}"  
      method="get"  
      role="search">  
  <label for="q">搜尋商品</label>  
  <input id="q" name="q" value="{ { query } }">  
  <button type="submit">搜尋</button>  
</form>
```

送出後瀏覽器形成：

/?q=鍵盤

`name="q"` 決定 query string key；GET 適合可分享、可 bookmark、重複執行的查詢。

12-9 Query string 由 request.GET 讀取

```
query = request.GET.get("q", "").strip()
```

- request.GET 是 QueryDict-like object
- .get("q", "")：沒有 q 時使用空字串
- .strip()：移除前後空白
- GET value 仍是外部輸入，不能直接當可信 SQL 或任意欄位名稱

接著才決定是否加 filter：

```
if query:  
    queryset = queryset.filter(...)
```

12-10 搜尋名稱或說明

```
from django.db.models import Q

if query:
    queryset = queryset.filter(
        Q(name__icontains=query) |
        Q(description__icontains=query)
    )
```

- 空 query 不加搜尋條件
- `icontains` 由 ORM 參數化處理 value
- `Q | Q` 表示名稱或說明任一符合
- QuerySet 繼續保持可組合

這一頁只解釋 value filter；若讓使用者選任意排序欄位，仍需要 allowlist。

12-11 把搜尋字符串送回 template

```
return render(request, "marketplace/home.html", {  
    "products": queryset,  
    "query": query,  
})
```

```
<input name="q" value="{{ query }}">  
{% if query %}  
    <p>「{{ query }}」的搜尋結果</p>  
{% endif %}
```

這叫 state retention：request 進來的查詢狀態，經 View 正規化後再回到 UI。

Template autoescaping 會處理 query 的 HTML 特殊字元；但仍要由 ORM 安全處理 database 查詢。

12-12 分類篩選同樣是 GET state

```
category = request.GET.get("category", "").strip()
if category:
    queryset = queryset.filter(category__slug=category)
```

Context 需要：

```
{
    "categories": Category.objects.all(),
    "selected_category": category,
}
```

Template 用這兩個值列出分類、標示目前分類，並建立保留搜尋條件的連結。

12-13 步驟 A：多個 query parameters 連結語法

目前 LearnMart 節錄／重排 | category link 的概念

```
<a href="?category={{ category.slug }}{% if query %}&q={{ query|urlencode }}{% endif %}">
  {{ category.name }}
</a>
```

- 第一個參數前使用 `?`
- 後續參數用 `&`
- `urlencode` 把空白與特殊字元編碼成合法 URL component

若切分類時忘記 `q`，搜尋條件會消失；若分頁忘記 `category`，分類條件也會消失。

12-14 步驟 B：辨識目前 UI 的狀態保持邊界

目前 LearnMart 實作：

- category links 保留 q
- pagination links 保留 q/category
- navbar search form 目前只提交 q，所以重新搜尋會重設 category

這不是安全規則，而是 UI navigation contract。

配套實作手冊： low-stock lab 會使用 `{% querystring %}`，並把「search 同時保留 category」列為 intentional improvement。

12-15 商品卡會讀 relation，所以先載入

目前 LearnMart 節錄／重排 | `marketplace/views.py` 的 `ProductListView.get_queryset()`

```
queryset = Product.objects.filter(  
    is_active=True,  
).select_related("category", "seller")
```

目前 LearnMart 節錄 | `templates/marketplace/home.html`

```
{{ product.category.name }}  
{{ product.seller.username }}
```

Query 與 template 應一起讀：看見 template 取 relation，就回頭確認 QuerySet 是否造成 N+1。

12-16 圖片、fallback 與 alt 要在同一段理解

```
{% if product.image %}
    
{% else %}
    <div class="placeholder-product">商品圖片</div>
{% endif %}
```

- ImageField 為空時，不可直接取 `.url`
- 有圖時以商品名稱作為替代文字
- 無圖時提供可見 fallback
- 圖片 storage/media routing 必須能提供 URL

第 15 章介紹上傳驗證，第 21 章整合上傳安全。

12-17 空結果不是錯誤，是正常 UI state

```
{% for product in products %}
...商品卡...
{% empty %}
<div class="alert alert-light border">
  目前找不到符合條件的商品。
</div>
{% endfor %}
```

- 搜尋 0 筆仍應回 200
- 使用者需要知道「查詢成功但沒有結果」
- 這和 404 不同：404 表示路由／單一物件找不到
- Empty state 應保留搜尋欄與分類控制，讓使用者調整條件

第 12 章實作與離堂檢核

任務： 用 Function View 追蹤列表與詳情，完成搜尋、分類及空結果檢核。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 01 原第 6 章](#)；[LearnMart 01 原第 6 章](#)。手冊保留原章號，對照表列出本課位置。

第 13 章

CBV 與列表整合

以 ListView／DetailView 解釋相同流程，驗證搜尋、分類、分頁能一起工作。

本章的操作環境與成果

前台實作選 LearnBoard 或 LearnMart；圖解與最小範例先說明概念，再對照現有程式。

完成成果：以 ListView／DetailView 解釋相同流程，驗證搜尋、分類、分頁能一起工作。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

13-1 先補 Django class 需要的 Python 語法

```
class ProductListView(ListView):  
    model = Product  
    paginate_by = 12
```

- `ProductListView` 繼承 `ListView`
- `model`、`paginate_by` 是 class attributes，用來設定行為
- 每個 request 會建立 view instance
- `self` 指向本次 request 使用的 instance

13-2 Override：改寫繼承來的方法

```
def get_queryset(self):  
    return Product.objects.filter(is_active=True)
```

`ListView` 原本已知道如何取得 `QuerySet`；`override` 讓你替換資料範圍。

方法名稱與 `return type` 必須符合 `generic view contract`；回傳 `None` 會讓後續流程失敗。

13-3 `super()`：保留父類別既有工作

```
def get_context_data(self, **kwargs):  
    context = super().get_context_data(**kwargs)  
    context["categories"] = Category.objects.all()  
    return context
```

`super()` 不是「跳過目前 class」，而是依繼承順序呼叫下一個實作。先取得 Django 已準備的 pagination/object list context，再增加自己的 key。

13-4 *args 與 **kwargs 在 Django 常出現

```
def __init__(self, *args, **kwargs):  
    super().__init__(*args, **kwargs)
```

- *args 收集額外 positional arguments
- **kwargs 收集額外 keyword arguments
- 轉交給父類別，可保留 framework 的彈性

初學時不必背所有參數，但不要把它們刪掉而破壞父類別初始化。

13-5 為什麼 URLconf 不能直接放 CBV class ?

URLconf 需要「接 request 並回 response 的 callable」。

```
path("", ProductListView.as_view(), name="home")
```

`.as_view()` 會回傳 callable；每次 request 時：

1. 建立 view instance
2. 設定 `request`、`args`、`kwargs`
3. 依 HTTP method dispatch 到 `get()` 或 `post()`

13-6 CBV request 流程簡圖

```
URLconf: ProductListView.as_view()  
→ setup(request, ...)  
→ dispatch()  
→ get()  
→ get_queryset()  
→ paginate_queryset()  
→ get_context_data()  
→ render_to_response()
```

通常只 override 需要改的 hook，不要重寫整條流程。

13-7 目前 LearnMart ProductListView

目前 LearnMart | 逐字摘錄 | marketplace/views.py::ProductListView

```
class ProductListView(ListView):  
    model = Product  
    template_name = "marketplace/home.html"  
    context_object_name = "products"  
    paginate_by = 12
```

明確設定 `template_name`，因為專案沒有使用預設的 `marketplace/product_list.html`。

13-8 `get_queryset()` 決定可見資料

```
def get_queryset(self):  
    queryset = Product.objects.filter(is_active=True).select_related(  
        "category", "seller"  
    )  
    ...  
    return queryset
```

這裡同時定義：

- 只有上架商品
- 預先載入 category/seller
- 後續依 GET 搜尋與分類縮小結果

QuerySet 是資料可見性的第一道門。

13-9 `get_context_data()` 增加模板需要的資料

```
def get_context_data(self, **kwargs):  
    context = super().get_context_data(**kwargs)  
    context["categories"] = Category.objects.all()  
    context["query"] = self.request.GET.get("q", "")  
    context["selected_category"] = self.request.GET.get("category", "")  
    return context
```

`self.request` 是目前 request；`super()` 已加入 `products`、`pagination` 等 context。

13-10 DetailView：單筆查詢仍可縮小 queryset

```
class ProductDetailView(DetailView):  
    model = Product  
    template_name = "marketplace/product_detail.html"  
    context_object_name = "product"  
  
    def get_queryset(self):  
        return Product.objects.filter(is_active=True)
```

URL 的 `pk` 不在這個 queryset 中時，自動得到 404。

13-11 Pagination 也是 CBV 的既有能力

```
class ProductListView(ListView):  
    paginate_by = 12
```

Template 得到 `page_obj`、`is_paginated` 等 context。

若搜尋與分類使用 query string，分頁連結要保留它們，否則點第二頁會丟失目前條件。

13-12 CBV 常見錯誤

檢查順序

- URL 忘記 `.as_view()`
- `template_name` 指到不存在檔案
- `get_queryset()` 沒有 return
- `get_context_data()` 沒先呼叫 `super()`
- 只檢查 role，沒有 ownership
- `form_valid()` 補 seller 後沒 return `super()`
- mixin 放在 generic view 後，行為不如預期

13-13 同一個 vertical slice，資料領域不同

以下是目前 `learnboard/board/views.py` 的真實查詢骨架；它和前面 LearnMart 的 `ProductListView` 使用同一個 CBV／QuerySet／context 模式。

```
class MessageListView(ListView):
    model = Message
    template_name = "board/message_list.html"
    context_object_name = "posts"
    paginate_by = 10

    def get_queryset(self):
        queryset = Message.objects.select_related("author")
        query = self.request.GET.get("q", "").strip()
        if query:
            queryset = queryset.filter(Q(content__icontains=query))
        return queryset
```

13-14 LearnBoard 與 LearnMart 的列表差異

LearnBoard	LearnMart
posts	products
只查 content	查 name 或 description ，另加 category
select_related("author")	select_related("category", "seller")

13-15 專案對照：用資料規則解讀留言牆

運用第 8～9 章學過的關聯與 migration，回查兩個專案的實際檔案。

比較項目	LearnBoard	LearnMart
頁面主要資料	Message	Product
使用者角色	留言作者 author	商品賣家 seller
作者／帳號演進	0002 加入可空的 author	初始 migration 已使用自訂 User
頁面需考慮的情境	沒有作者時如何顯示？	商品如何顯示分類與賣家？

任務： 指出留言模板處理空作者的位置，再追蹤 View 如何載入作者資料。
關聯規則回查第 8 章；migration 演進回查第 9 章。

13-16 LearnBoard 的搜尋與模板輸出

```
def get_context_data(self, **kwargs):
    context = super().get_context_data(**kwargs)
    context["query"] = self.request.GET.get("q", "")
    return context
```

```
{% for post in posts %}
  <article class="card">
    <p>{{ post.content }}</p>
    <small>{{ post.author|default:"訪客" }}</small>
  </article>
{% empty %}
  <p>目前沒有符合條件的留言。</p>
{% endfor %}
```

同樣的資料流在 LearnMart 會變成商品卡、分類連結、圖片 fallback 與共用 pagination partial；核心仍是「View 準備 context，Template 負責呈現」。

13-17 步驟 A：從 Function View 收斂至 LearnMart

目前 LearnMart 節錄 | ProductListView

```
class ProductListView(ListView):  
    model = Product  
    template_name = "marketplace/home.html"  
    context_object_name = "products"  
    paginate_by = 12
```

- `get_queryset(self) -> QuerySet`：active、搜尋、分類、select_related
- `self.request`：目前 HttpRequest
- `get_context_data(self, **kwargs) -> dict`：加入 categories 與 query state
- `super().get_context_data(**kwargs)`：保留 parent 已建立的 pagination context
- 兩個 overrides 都必須 `return` 結果

把本章已學的 `.as_view()`、繼承與 override 流程套回這個 class。

13-18 步驟 B：目前 source 的 query state 優化細節

目前 LearnMart 節錄 | `get_context_data()`

```
context["query"] = self.request.GET.get("q", "")
context["selected_category"] = self.request.GET.get(
    "category", "",
)
```

Filter 端使用 `.strip()`，但 context 端再次讀 raw value。因此 `q=" "` 不會 filter，UI 卻可能把空白視為有搜尋字串。

配套實作手冊的 intentional improvement：把送回 UI 的 q/category 也 `.strip()`；這不是聲稱目前 source 已正規化。

13-19 Pagination 是 QuerySet 與 UI 的共同狀態

目前最終 `ProductListView` 設定：

```
paginate_by = 12
```

Template 會得到：

- `page_obj`：目前頁
- `paginator`：總頁數、總筆數
- `is_paginated`

Pagination link 不只要改 `page`，還要保留 `q/category`：

```
?page=2&q=鍵盤&category=tech
```

分頁連結保留搜尋與分類；改變條件時移除舊 `page`，回到第 1 頁。

13-20 `include` 適合抽出分頁 UI

目前 LearnMart 節錄 | `templates/marketplace/home.html`

```
{% include "marketplace/pagination.html" %}
```

目前 partial 直接使用 `page_obj`、`query`、`selected_category`；但它也被留言板共用，因此這些 catalog-only names 會讓 reusable boundary 變窄。

補充／進階 | Django 6.1.1 generic query preservation

```
<a href="{% querystring page=page_obj.next_page_number %}">  
    下一頁  
</a>
```

`querystring` 會從目前 request query parameters 產生新 URL，只替換 page；可自然保留 q/category/low_stock，也避免共享 partial 硬綁 catalog context。

13-21 步驟 A：Vertical slice 前半（Request 轉 QuerySet）

```
GET /?q=鍵盤&category=tech
|
▼
config/urls.py
| include("marketplace.urls")
|
▼
marketplace/urls.py
| home → ProductListView
|
▼
Product QuerySet
active + search + category + select_related
```

這一半的輸入是 HTTP request；輸出是尚可繼續組合、之後才評估的 QuerySet。

13-22 步驟 B：Vertical slice 後半（Context 轉 HTML）

QuerySet + categories + query state



View 建立 context 並 pagination



templates/marketplace/home.html



extends / include



200 HTML

商品卡 + 分類 + 搜尋字串 + 分頁

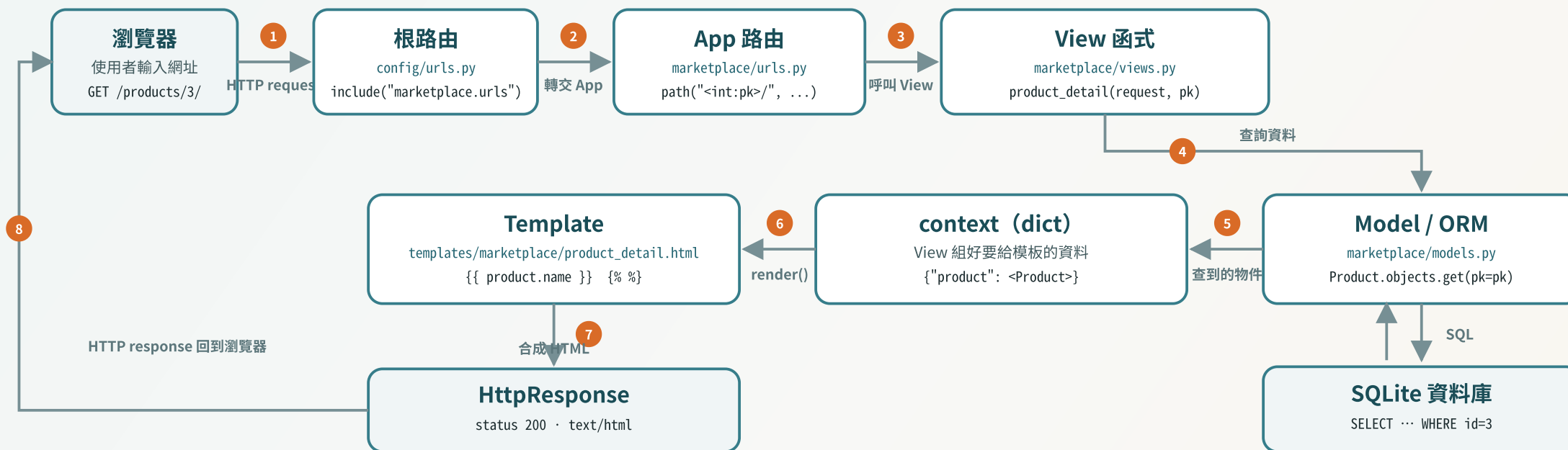
每層只負責自己的部分，但 context key、template variable 與 URL state 契約必須對得上。

13-23 步驟 A：完整 request flow 逐層解析

1. Browser：GET `/?q=鍵盤`
2. URLconf：名稱 `marketplace:home` 指向 list view callable
3. View：從 `request.GET` 取得 q
4. ORM：組成 active + `Q(...)` QuerySet
5. View：加 categories/query context，執行 pagination
6. Template：loop products、反向 URL、讀 relation
7. Bootstrap/static：形成 responsive 商品卡
8. Django：回 200 HTML response

若你能指出每一步的檔案與資料型別，就不是只會複製程式碼。

13-24 步驟 B：圖解 Catalog 的完整資料流



每一格都對應一個檔案；出錯時先問：「卡在第幾步？」

13-25 步驟 A：手動驗證 catalog 行為

- `/` 顯示 seeded 商品
- `/?q=鍵盤` 只顯示符合名稱／說明的商品
- category links 可保留 q；global search 目前會重設 category
- 搜尋／分類改變時回到第 1 頁；pagination 保留查詢條件
- 空結果顯示正常 empty state 且 status 仍為 200
- detail link 由命名 URL／`get_absolute_url()` 產生
- 手機、平板、桌面欄數按 breakpoint 改變

你應該看到：每一項都能指出 URL、頁面文字或 viewport 變化，而不是只說「看起來可以」。

13-26 步驟 B：用 focused test 固定搜尋行為

目前 LearnMart 實作 | 單一 test label

```
uv run python manage.py test \  
    "marketplace.tests.MarketplaceFlowTests.test_search_finds_product"
```

預期：

```
Ran 1 test  
OK
```

反斜線只延續 command；完整 dotted label 維持在同一個 quoted shell argument。完整 test suite 仍使用 `uv run python manage.py test`。

13-27 觀念檢核與實作

1. GET form 的 `name="q"` 如何一路變成 ORM filter ?
2. 搜尋＋分類＋分頁時，為什麼要保留多個 query parameters ?
3. 商品卡讀 category/seller 時，為何使用 `select_related` ?
4. 空 QuerySet 為什麼通常回 200 而不是 404 ?
5. 為什麼 low-stock threshold 與可查欄位必須由 server 定義 ?

實作任務： 增加「低庫存」GET filter；變更 q/category/low_stock 時重設 page，pagination 則保留三個 filter states，並提供空結果與 boundary test。

配套實作手冊： LearnMart [原第 6 章](#)；LearnBoard [類比原第 6 章](#)

13-28 兩個首頁目前各自做了哪些事？

這一頁只描述目前完成專案，不是要你立刻仿寫所有 class-based View。

首頁責任	LearnBoard	LearnMart
View	MessageListView	ProductListView
基礎資料	Message，並預先取得 author	啟用的 Product，並預先取得列表卡片所需關聯
URL 狀態	q：留言文字搜尋	q：文字搜尋；category：分類篩選
Template	board/message_list.html	marketplace/home.html
列表 UI	留言卡、作者、時間、分頁	商品卡、圖片 fallback、分類、分頁

兩者都是「URL → View 讀取 request.GET → QuerySet → context → template」；差別在領域資料與篩選規則。

第 3 到第 12 章再拆開 Template、Model、ORM 與列表頁的細節。

13-29 可讀取資料的網站：整段驗收

1. 從 URL 找到 View，指出 QuerySet 與 context 的名稱。
2. 用 Admin 修改一筆資料，在前台確認顯示結果。
3. 組合搜尋、分類與分頁，檢查條件是否保留。
4. 測試空結果、無效 pk 與無圖商品。
5. 指出 LearnBoard 與 LearnMart 共用的骨架及不同資料規則。

把驗收紀錄與對應測試保存在自己的練習分支。

13-30 接下來：從查詢走向安全寫入

- 第 14 章先整理顯示工具與 Template 元件。
- 第 15 章讓表單驗證資料、顯示錯誤並保存結果。
- 第 16～17 章加入登入、角色及物件擁有權。
- 第 18～20 章把相同機制組成購物車、訂單與評價。

查詢頁面的 URL、ORM、context 與模板分工仍然沿用。

第 13 章實作與離堂檢核

任務： 以 ListView／DetailView 解釋相同流程，驗證搜尋、分類、分頁能一起工作。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： LearnBoard 01 原第 6 章；LearnMart 01 原第 6 章。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[08_DTL進階與元件整理](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。